

Kubernetes 総まとめ

構築 → 通信 → 配置 → Storage → Security → HPA → 障害診断 → EKS

学習の軸

作る → つなぐ → 置く → 守る → 残す → 増やす → 更新する → 壊れたら診断する

kind 3-Node + frontend + Go backend + PostgreSQL を使った実験ベース

目次

- 1. まず全体像 - Kubernetes は何を分担しているか
- 2. Cluster / Control Plane / Node / Pod
- 3. Workload - Pod をどう管理するか
- 4. 通信 - Ingress / Service / EndpointSlice / DNS
- 5. Scheduling - Pod をどの Node に置くか
- 6. Storage - Pod の寿命とデータの寿命を分ける
- 7. ConfigMap / Secret / Namespace
- 8. Security - NetworkPolicy / ServiceAccount / RBAC
- 9. Autoscaling - Metrics Server / HPA / Node
- 10. Rolling Update / Rollback
- 11. kubectl - 目的から逆引き
- 12. 障害診断フロー完全版
- 13. 実験で確認した障害パターン
- 14. kind から EKS へ
- 15. 本番運用で追加するもの
- 16. 用語辞典
- 17. 最終チートシート

この資料の使い方 暗記用ではなく「症状や目的から次に何を見るか」を辿るための地図として使う。

1. まず全体像 - Kubernetes は何を分担しているか

Kubernetes

- ― 何を動かす？ → Deployment / StatefulSet / DaemonSet / Job / CronJob
- ― どうつなぐ？ → Service / EndpointSlice / Ingress / CoreDNS
- ― どこに置く？ → Scheduler / Node / requests / Affinity / Taint / Spread
- ― どう守る？ → Secret / ServiceAccount / RBAC / NetworkPolicy
- ― どう残す？ → PVC / PV / StorageClass
- ― どう増やす？ → Metrics Server / HPA
- ― 正常か？ → readiness / liveness / startup probe
- ― どう更新する？ → Rolling Update / Rollback
- ― 壊れたら？ → get → describe → logs → 通信/metrics/RBACを確認

一番大事な見方 Kubernetesは巨大な1機能ではなく、複数のControllerとResourceが「望ましい状態」を分担して維持する仕組み。

2. Cluster / Control Plane / Node / Pod

Kubernetes Cluster

- ― Control Plane 「全体を管理」
 - ― API Server ← kubectlやControllerの入口
 - ― etcd ← Cluster状態を保存
 - ― Scheduler ← Podの配置先Nodeを決定
 - ― Controller Manager ← Desired Stateへ近づける
- ― Worker Node 「Podを実際に動かすサーバー」
 - ― CPU / Memory / Storage / Network
 - ― kubelet ← Podを起動・監視
 - ― Pods

用語	イメージ	役割
Cluster	サーバー群全体	Kubernetesで管理する単位
Node	サーバー1台	CPU・メモリ等を持つPodの置き場所
Pod	実行単位	1つ以上のContainerをまとめる最小単位
Container	プロセス実行環境	実際のアプリプロセスを動かす

kind-multi (ローカル)

Mac / OrbStack

- ― multi-control-plane
- ― multi-worker
- ― multi-worker2

EKS (本番イメージ)

AWS

- ― Managed Control Plane
- ― EC2 Node
- ― EC2 Node

3. Workload - Pod をどう管理するか

Pod を動かしたい

↓

ずっと動く？

└─ NO

└─ 定期実行？ YES → CronJob

└─ 1 回だけ？ → Job

└─ YES

└─ 各 Node に 1 個？ → DaemonSet

└─ それ以外

└─ 固定 identity / 個別 Storage 必要 → StatefulSet

└─ 入れ替わってよい → Deployment

Workload	使うとき	Pod の性質
Deployment	Web/API など常駐 Stateless	交換可能。ReplicaSet 経由で Pod 数を維持
StatefulSet	DB や固定 identity が重要	固定名・順序・個別 PVC
DaemonSet	各 Node に Agent を置く	対象 Node ごとに原則 1Pod
Job	1 回処理して終了	Completed が正常
CronJob	定期実行	CronJob → Job → Pod

Deployment → ReplicaSet → Pod

StatefulSet → postgres-0 / postgres-1 ...

DaemonSet → Node A に 1Pod / Node B に 1Pod

Job → Pod → Completed

CronJob → Job → Pod → Completed

4. 通信 - Ingress / Service / EndpointSlice / DNS

Browser

→ Ingress Controller

→ Ingress

→ frontend Service

→ EndpointSlice

→ frontend Pod (nginx)

→ backend Service

→ EndpointSlice

→ backend Pod

→ postgres Service

→ postgres-0

要素	役割	見るポイント
Ingress	HTTP/HTTPS の入口ルール	path/host → Service
Service	Pod 群への固定入口	selector / port / targetPort
EndpointSlice	Service の実転送先	endpoint / ready / port
CoreDNS	Service 名の名前解決	Service 名 / Namespace / FQDN
NetworkPolicy	Pod 間通信の許可/拒否	podSelector / ingress / egress

Service 名で通信

backend Pod

→ "postgres"

→ CoreDNS

→ postgres Service

→ postgres-0

FQDN: <service>.<namespace>.svc.cluster.local

通信障害の基本順序 DNS → Service → EndpointSlice → ready → port/targetPort → NetworkPolicy → app listen の順で追う。

5. Scheduling - Pod をどの Node に置くか

Pod 作成要求

→ Scheduler

├ requests で CPU/Memory 足りる？

├ nodeSelector 合う？

├ Affinity / Anti-Affinity 合う？

├ Taint を Toleration できる？

└ topologySpreadConstraints 守れる？

→ Node 決定

→ kubelet

→ Pod Running

要素	意味
requests	Scheduler が必要資源として予約・計算する基準
limits	Container が使える上限。CPU は throttle、Memory は OOM の可能性
nodeSelector	指定ラベル Node だけに置く
Affinity	指定条件へ寄せる/必須にする
Anti-Affinity	Pod 同士を離す
Taint	Node 側の「基本来ないで」
Toleration	Pod 側の「その Taint でも置いてよい」
topologySpreadConstraints	Node/AZ 間で Pod 数の偏りを抑える

HPA で 6Pod

→ Scheduler

→ topologySpreadConstraints

→ multi-worker : 3Pod

→ multi-worker2 : 3Pod

実験で踏んだ罠 DoNotSchedule + 強い分散制約 + RollingUpdate の maxSurge がぶつくと新 Pod を置けず rollout が止まる。配置制約と更新戦略はセットで設計する。

6. Storage - Pod の寿命とデータの寿命を分ける

Pod
→ volumeMount
→ PVC 「1Gi、RW0 の領域ください」
→ PV 「実際の保存領域」
→ Storage

要素	役割
PVC	Pod 側からの Storage 要求
PV	Cluster 側の実 Storage 表現
StorageClass	PVC から PV をどう Provision するか
volumeMount	Container 内のどこへ mount するか
CSI	Cloud Storage 等と Kubernetes をつなぐ Driver

今回の PostgreSQL
StatefulSet
→ postgres-0
→ /var/lib/postgresql/data
→ postgres-data-postgres-0 (PVC)
→ PV
→ 保存

postgres-0 削除
→ StatefulSet が再作成
→ 同じ PVC 再利用
→ messages データが残る ✓

EKS
Pod → PVC → PV → EBS CSI → EBS
共有 Storage なら → EFS CSI → EFS

7. ConfigMap / Secret / Namespace

コードから環境差を外に出す
└─ 普通の設定 → ConfigMap
└─ 機密値 → Secret
└─ Resource 群の論理区切り → Namespace

要素	例	注意
ConfigMap	DB_HOST / DB_NAME	機密値は入れない
Secret	DB_USER / DB_PASSWORD	Base64 は暗号化ではない
Namespace	dev/staging/prod	Network 隔離そのものではない

Namespace: production
└─ ConfigMap postgres-config
└─ Secret postgres-secret
└─ frontend Deployment
└─ backend Deployment
└─ postgres StatefulSet

本番 **Secret** RBAC ・ etcd encryption at rest ・ AWS Secrets Manager 等の外部 Secret 管理を組み合わせる。

8. Security - NetworkPolicy / ServiceAccount / RBAC

値を守る → Secret
API 上の身元 → ServiceAccount
API 操作権限 → Role / ClusterRole + Binding
Pod 間通信 → NetworkPolicy
作成できる設定 → Admission Policy 等

backend Pod
→ ServiceAccount: backend-sa
→ RoleBinding
→ Role: pods get/list/watch

list pods 
delete pods 
list deployments 

NetworkPolicy
frontend → backend 
backend → postgres 
その他 → backend 
frontend → postgres 

重要 NetworkPolicy は「本物の backend か」を認証しない。ラベルで判定する。fake-backend に app=backend を付けると postgres へ通れたため、RBAC や Admission 等と組み合わせる必要がある。

9. Autoscaling - Metrics Server / HPA / Node

負荷
→ kubelet metrics
→ Metrics Server
→ Metrics API
→ HPA
→ Deployment replicas 変更
→ ReplicaSet
→ Pod 増減
→ Scheduler
→ Node へ配置

backend requests.cpu = 50m
HPA target = 50%
→ 1Pod あたり約 25m を目標

実験
2Pod → CPU 134% → 4 → 6Pod
負荷停止 → 低 CPU → 数分維持 → 6 → 3 → 2Pod

HPA の% Node 全体の CPU%ではなく、Pod の CPU request に対する使用率。requests.cpu がない対象 Pod が混ざると <unknown>になることがある。

EKS 本番では二段階
HPA → Pod を増やす
↓ Node 容量不足
Karpenter / Cluster Autoscaler → Node を増やす

10. Rolling Update / Rollback

Deployment 変更
→ 新 ReplicaSet
→ 新 Pod 作成
→ readiness 成功
→ 旧 Pod 削除
→ 徐々に入れ替え

設定	意味
maxSurge	更新中に replicas より何個余分に増やせるか
maxUnavailable	更新中に何個まで Unavailable を許すか
readinessProbe	新 Pod を Service/rollout 上で使えるか判定
rollout undo	前の Pod template へ戻す

強い配置制約がある今回の設定例

strategy:

```
type: RollingUpdate
rollingUpdate:
  maxSurge: 0
  maxUnavailable: 1
```

kubectl rollout status deployment/backend

kubectl rollout history deployment/backend

kubectl rollout undo deployment/backend

11. kubectl - 目的から逆引き

目的	コマンド	意味
状態を見る	kubectl get ...	広く浅く
詳細/Events	kubectl describe ...	1Resource を深く
アプリログ	kubectl logs POD	標準出力/標準エラー
直前のログ	kubectl logs POD --previous	前回 Container のログ
中で実行	kubectl exec -it POD -- sh	Pod 内調査
作成/更新	kubectl apply -f file.yaml	Desired State 反映
削除	kubectl delete ...	Resource 削除
更新監視	kubectl rollout status ...	Rollout 完了待ち
負荷	kubectl top pods/nodes	Metrics を見る
権限	kubectl auth can-i ...	RBAC 確認
Context	kubectl config current-context	今どの Cluster か

まず get
↓
詳しく describe
↓
アプリなら logs
↓
内部確認なら exec

通信: svc → endpointslice → pod → networkpolicy

配置: Pending → describe → Events

負荷: top / describe hpa

権限: auth can-i

12. 障害診断フロー完全版

```
kubectll get pods
|
|— Pending
|   → describe / Events
|   → CPU/Memory 不足・Taint・Selector・Affinity・Spread
|
|— ImagePullBackOff
|   → describe / Events
|   → image/tag/registry/認証
|
|— CrashLoopBackOff
|   → logs → logs --previous → describe
|   → app crash / DNS / 設定 / OOM など
|
|— Running 0/1
|   → readinessProbe
|   → EndpointSlice ready=false
|
|— Running + RESTARTS 増加
|   → liveness / OOM / app crash
|
|— Running 1/1
|   → 通信系を確認
|       DNS → Service → EndpointSlice → port → NetworkPolicy → app listen
```

通信できない

↓

名前解決？

└─ NO → CoreDNS / Service 名 / Namespace

└─ YES

↓ Service あり？

↓ EndpointSlice あり？

└─ endpoints: null → Service selector vs Pod labels

└─ ready=false → readiness

└─ ready=true

↓ targetPort / 実 listen port

↓ NetworkPolicy

↓ app logs

13. 実験で確認した障害パターン

症状	実験原因	診断の主役	確認結果
ImagePullBackOff	存在しない image tag	describe / Events	pull access denied / repository does not exist
Running 0/1	readiness path を 404 へ	describe + EndpointSlice	ready=false, serving=false
RESTARTS 増加	liveness path を 404 へ	describe / Events	failed liveness probe, will be restarted
CrashLoopBackOff	DB_HOST=postgres-broken	logs	CoreDNS no such host
OOMKilled	32Mi limit で連続確保	describe	Reason OOMKilled / Exit 137
CPU 頭打ち	limit 50m で busy loop	kubectl top	約 51m、Pod は Running
timeout	NetworkPolicy selector を壊す	NetworkPolicy	frontend→backend が drop
timeout	Service targetPort=9999	Service/EndpointSlice	Pod は 8080 listen
502	Service selector 不一致	EndpointSlice	endpoints: null
HPA <unknown>	fake-backend に requests.cpu なし	describe hpa	FailedGetResourceMetric
Rollout 停止	DoNotSchedule + maxSurge	Pending Pod describe	topology spread 制約

覚え方 同じ「API が失敗」でも症状だけで決め打ちしない。Pod → Service → EndpointSlice → Policy → port → app の順で事実を集める。

14. kind から EKS へ

kind / Kubernetes	EKS / AWS	考え方
kind Cluster	EKS Cluster	Kubernetes API は同じ
kind Node	EC2 / Managed Node Group / Fargate	Pod の実行場所
Ingress Controller	AWS Load Balancer Controller	Ingress から ALB/NLB を実装
PVC/PV	EBS/EFS + CSI	Storage の実体が AWS
Secret	Kubernetes Secret + Secrets Manager	機密値管理を強化
HPA	HPA	基本同じ
Node 不足	Karpenter / Cluster Autoscaler	Node 自体を増減
RBAC	RBAC + IAM/Pod Identity	K8s 権限と AWS 権限は別
NetworkPolicy	NetworkPolicy + SG/NACL	K8s と AWS Network の 2 層
kubectl logs/top	CloudWatch/Prometheus/Grafana 等	継続監視を追加

Internet
 → Route 53
 → ALB
 → AWS Load Balancer Controller
 → Ingress
 → frontend Service → Pods
 → backend Service → Pods
 → RDS / Aurora

ポイント kind で学んだ Deployment / Service / Ingress / HPA / RBAC / NetworkPolicy / PVC / Probe は EKS でも残る。下のインフラ実装と運用サービスが AWS になる。

15. 本番運用で追加するもの

領域	ローカル学習	本番で追加
可用性	2 Worker 分散	複数 AZ、PDB、Node/AZ spread
監視	get/describe/logs/top	Metrics/Logs/Tracing/Alert
Secret	Kubernetes Secret	Secrets Manager、暗号化、rotation
DB	StatefulSet PostgreSQL	RDS/Aurora を検討
Autoscaling	HPA	Karpenter/Cluster Autoscaler
Delivery	kubectl apply	CI/CD、GitOps、Canary/Blue-Green
Policy	RBAC/NetworkPolicy	Admission、Pod Security、Image policy
Backup	PVC 実験	DB backup、Volume snapshot、restore 訓練

Production readiness の問い

- └─ Node/AZ が 1 つ落ちても動く？
- └─ Secret が漏れにくい？ rotation できる？
- └─ Pod が増えて Node 容量を超えたら？
- └─ 更新失敗時に自動/手動で戻せる？
- └─ 5xx / latency / restart を検知できる？
- └─ DB/Storage を restore できる？
- └─ 誰が何を変更できるか追跡できる？

16. 用語辞典

用語	意味
API Server	Kubernetes API の入口。kubectl や Controller がここへアクセスする。
etcd	Cluster の状態を保存する分散 KVS。
Controller	現在状態を Desired State へ近づけ続ける。
Reconciliation	差分を検知して望ましい状態へ戻すループ。
Pod	Kubernetes の最小実行単位。
ReplicaSet	指定数の同一 Pod を維持する。通常 Deployment が管理。
Deployment	Stateless な常駐 Pod の更新・Replica 管理。
StatefulSet	固定 identity・順序・個別 Storage を持つ Pod 管理。
DaemonSet	対象 Node ごとに Pod を配置。
Job	完了するまで処理する一回実行 Workload。
CronJob	スケジュールに従って Job を作る。
Service	Pod 群への安定した IP/DNS/Port の入口。
EndpointSlice	Service が転送可能な Endpoint 一覧。

Ingress	HTTP/HTTPS path/host routing を宣言。
Ingress Controller	Ingress を実際に処理する実装。
CoreDNS	Cluster 内 DNS。Service 名などを解決。
ConfigMap	非機密設定の外出し。
Secret	機密値の外出し。単体で金庫ではない。
Namespace	Cluster 内 Resource の論理区切り。
PVC	Storage を欲しい側の要求。
PV	実際の Persistent Storage 表現。
StorageClass	PV の動的 Provision 方法。
CSI	Storage provider を Kubernetes へ接続する標準。
requests	Scheduler/HPA の基準となる必要資源。
limits	Container の資源上限。
QoS	requests/limits 設定に基づく Pod のサービス品質クラス。
readinessProbe	トラフィックを受けてよいか。失敗しても通常再起動しない。
livenessProbe	Container を再起動すべきか。
startupProbe	起動完了まで liveness/readiness の猶予を与える用途。
HPA	Metrics に応じて replicas を自動調整。
Metrics Server	Pod/Node の resource metrics を提供。
Scheduler	Pod の配置先 Node を決める。
Taint	Node が Pod を拒む条件。
Toleration	Pod が Taint を許容する指定。
Affinity	Node/Pod への配置条件・好み。
Anti-Affinity	Pod 同士を離す配置条件。
topologySpreadConstraints	Node/AZ 間の Pod 数の偏りを制御。
ServiceAccount	Workload の Kubernetes API 上の identity。
Role	Namespace 内の API 権限セット。
RoleBinding	Role を User/Group/ServiceAccount へ付与。
ClusterRole	Cluster scope でも使える権限セット。
NetworkPolicy	Pod ingress/egress 通信の許可ルール。
Rolling Update	サービスを保ちながら Pod を段階的に更新。
Rollback	過去の Pod template へ戻す。
ImagePullBackOff	Container image 取得失敗後の backoff 状態。
CrashLoopBackOff	Container が起動と終了を繰り返す状態。
OOMKilled	Memory limit 等により OOM で kill された状態。
CPU throttling	CPU limit を超えようとした実行を抑制すること。

17. 最終チートシート

目的から考える

作る → Deployment / StatefulSet / DaemonSet / Job / CronJob
 つなぐ → Ingress / Service / EndpointSlice / DNS
 置く → Scheduler / requests / Taint / Affinity / Spread
 守る → Secret / ServiceAccount / RBAC / NetworkPolicy
 残す → PVC / PV / StorageClass
 増やす → Metrics Server / HPA
 正常性 → readiness / liveness
 更新 → Rolling Update / Rollback

障害が起きたら

1. `kubectl get pods`
2. Pending / ImagePull / CrashLoop / 0/1 / RESTARTS を分類
3. `kubectl describe pod`
4. 必要なら `kubectl logs / --previous`
5. 通信なら DNS → Service → EndpointSlice → port → NetworkPolicy → app
6. 負荷なら `kubectl top / describe hpa`
7. 権限なら `kubectl auth can-i`

症状 → 最初に見る場所

ImagePullBackOff → describe / Events

CrashLoopBackOff → logs / previous / describe

Running 0/1 → readiness / EndpointSlice

RESTARTS 増加 → liveness / OOM / app crash

Pending → describe / Scheduler Events

NXDOMAIN → CoreDNS / Service 名

endpoints:null → Service selector / Pod labels

HPA <unknown> → describe hpa / requests / Metrics Server

timeout → NetworkPolicy / port / 経路

Forbidden → ServiceAccount / RBAC / auth can-i

最後に Kubernetes は「Resource 名を覚える」より、「何を作る・どうつなぐ・どこに置く・どう守る・壊れたらどこを見る」を流れで持つと強い。